

Chapter 1: Introduction

Objectives

- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems

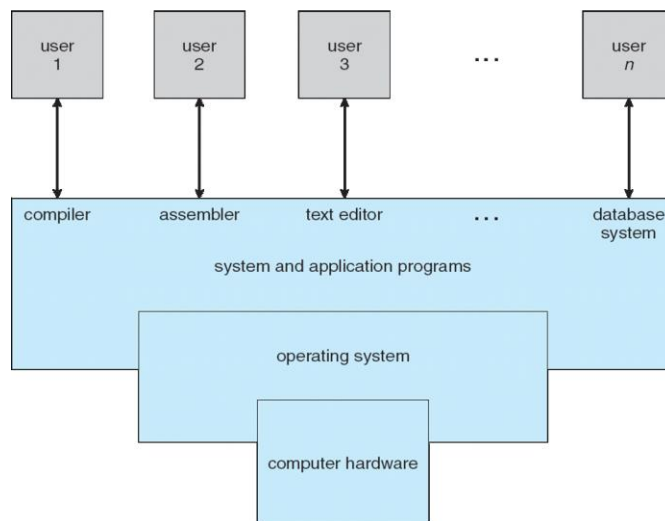
What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner

Computer System Structure

- ❖ Computer system can be divided into four components:
 - ✓ Hardware – provides basic computing resources
 - CPU, memory, I/O devices
 - ✓ Operating system
 - Controls and coordinates use of hardware among various applications and users
 - ✓ Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
 - ✓ Users
 - People, machines, other computers

Four Components of a Computer System



What Operating Systems Do

- ✓ Depends on the point of view
- ✓ Users want convenience, **ease of use** and **good performance**
 - Don't care about **resource utilization**
- ✓ But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- ✓ Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- ✓ Handheld computers are resource poor, optimized for usability and battery life
- ✓ Some computers have little or no user interface, such as embedded computers in devices and automobiles

Operating System Definition

- OS is a **resource allocator**
 - ✓ Manages all resources
 - ✓ Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - ✓ Controls execution of programs to prevent errors and improper use of the computer

Operating System Definition (Cont.)

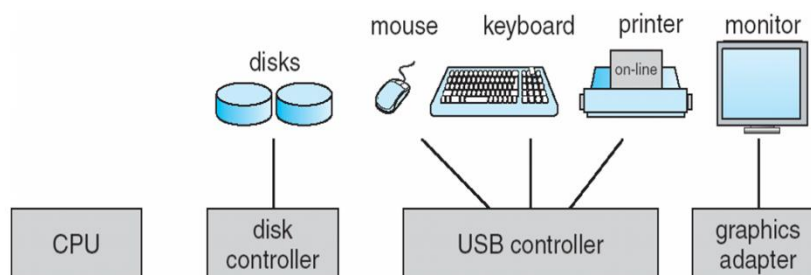
- ✓ No universally accepted definition
- ✓ “Everything a vendor ships when you order an operating system” is a good approximation
 - But varies wildly
- ✓ “The one program running at all times on the computer” is the **kernel**.
- ✓ Everything else is either
 - a system program (ships with the operating system) , or
 - an application program.

Computer Startup

- ✓ **bootstrap program** is loaded at power-up or reboot
 - Typically stored in ROM or EPROM, generally known as **firmware**
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution

Computer System Organization

- ✓ Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles



Computer-System Operation

- ✓ I/O devices and the CPU can execute concurrently
- ✓ Each device controller is in charge of a particular device type
- ✓ Each device controller has a local buffer
- ✓ CPU moves data from/to main memory to/from local buffers
- ✓ I/O is from the device to local buffer of controller
- ✓ Device controller informs CPU that it has finished its operation by causing an interrupt

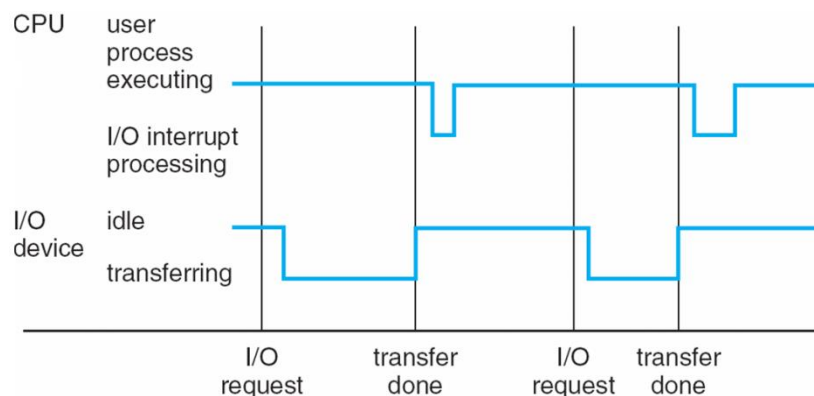
Common Functions of Interrupts

- ✓ Interrupt transfers control to the **interrupt service routine(ISR)** generally, through the **interrupt vector**, which contains the addresses of all the service routines
- ✓ Interrupt architecture must save the address of the interrupted instruction
- ✓ A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- ✓ An operating system is **interrupt driven**

Interrupt Handling

- ✓ The operating system preserves the state of the CPU by storing registers and the program counter
- ✓ Determines which type of interrupt has occurred:
 - **polling**
 - **vectored** interrupt system
- ✓ Separate segments of code determine what action should be taken for each type of interrupt

Interrupt Timeline



I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
 - ✓ Wait instruction idles the CPU until the next interrupt
 - ✓ Wait loop (contention for memory access)
 - ✓ At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
 - ✓ **System call** – request to the OS to allow user to wait for I/O completion
 - ✓ **Device-status table** contains entry for each I/O device indicating its type, address, and state
 - ✓ OS indexes into I/O device table to determine device status and to modify table entry to include interrupt

Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is $1,024^1$ bytes

a **megabyte**, or **MB**, is $1,024^2$ bytes

a **gigabyte**, or **GB**, is $1,024^3$ bytes

a **terabyte**, or **TB**, is $1,024^4$ bytes

a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

Storage Structure

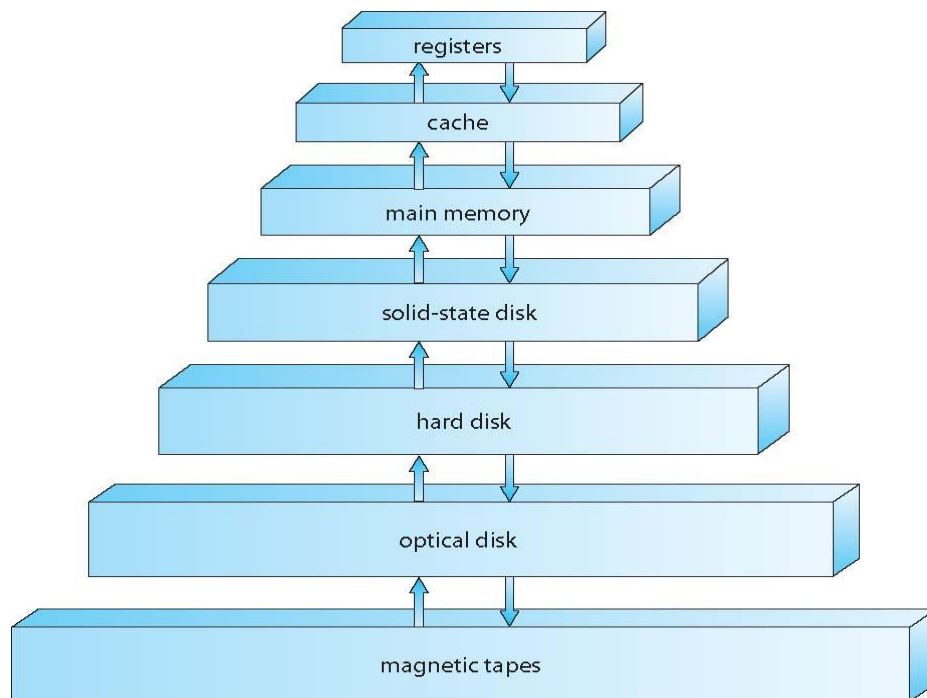
- ✓ Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically **volatile**
- ✓ Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- ✓ Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- ✓ **Solid-state disks** – faster than hard disks, nonvolatile

- Various technologies
- Becoming more popular

Storage Hierarchy

- ✓ Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- ✓ **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- ✓ **Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel

Storage-Device Hierarchy



Caching

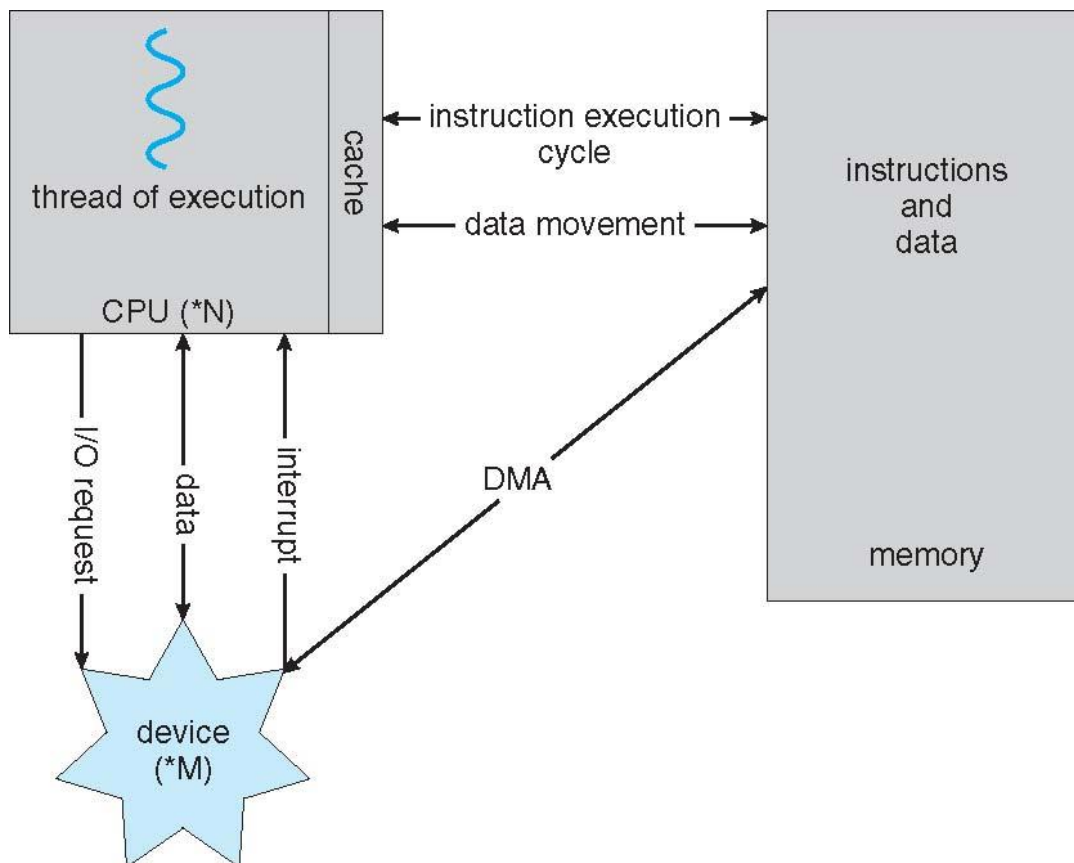
- ✓ Important principle, performed at many levels in a computer (in hardware, operating system, software)
- ✓ Information in use copied from slower to faster storage temporarily
- ✓ Faster storage (cache) checked first to determine if information is there

- If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- ✓ Cache smaller than storage being cached
- Cache management important design problem
 - Cache size and replacement policy

Direct Memory Access Structure

- ✓ Used for high-speed I/O devices able to transmit information at close to memory speeds
- ✓ Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- ✓ Only one interrupt is generated per block, rather than the one interrupt per byte

How a Modern Computer Works



Computer-System Architecture

- Most systems use a single general-purpose processor
 - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance
 - Two types:
 4. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
 5. **Symmetric Multiprocessing** – each processor performs all tasks

The First Operating System

- ▶ The first operating system used for real work was GM-NAA I/O, produced in 1956 by General Motors' Research division for its IBM 704. GM-NAA I/O was created in 1956 by Robert L. Patrick of General Motors Research and Owen Mock of North American Aviation
- ▶ OS/360 officially known as **IBM System/360 Operating System** based on batch processing system developed by IBM for their then-new System/360 mainframe computer, announced in 1964, was the first operating system that was developed

Disk Operating System (DOS)

- ▶ The first PC Operating System (1981)
- ▶ Not a Multitasking OS, only one program could run at a time
- ▶ A command-line interface, no GUI.
- ▶ Early versions of Windows sat on top of DOS and used it to communicate with the BIOS.
- ▶ Windows XP communicates directly with the BIOS, but allows commands to be entered via a command prompt.

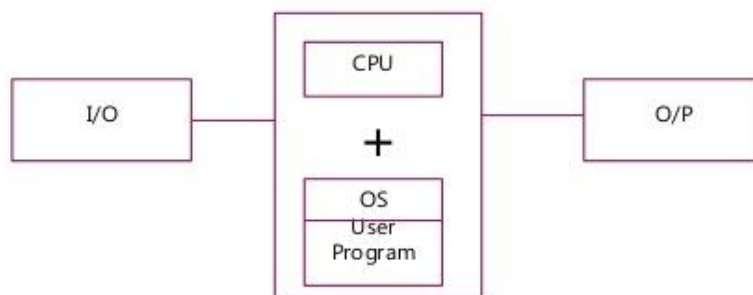
What came first: Windows or Linux?

- ▶ Windows 1.0 was released in 1985.
 - ▶ Linux kernel was first released in 1991.
-

Mainframe Computers

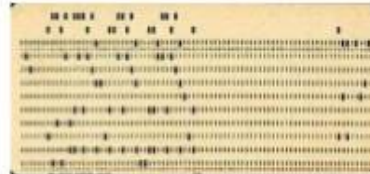
- ▶ Mainframes first appeared in the early 1940s.
- ▶ Vendors include IBM, Hitachi
- ▶ known for their large size, amount of storage, processing power and high level of reliability.
- ▶ Common I/O devices were card reader and tape drives

MainframeComputer



Punch Card

- ▶ Card of Thick Paper Based Material
- ▶ Used to contain digital data represented by the presence or absence of holes in predefined positions.
- ▶ When light passes we get 1/true
- ▶ Light doesn't pass we get 0/false
- ▶ Input is directly in the form of 0 or 1
- ▶ Used for simple/integers calculations
- ▶ For e.g:- Census Data Transactions



- ▶ User prepare a job which consist of program input output data and control instructions.
- ▶ Input output job is given in the form of punch card and result also appear in the form of punch card after processing.
- ▶ Operating system was very simple always present in memory.
- ▶ Major task is to transfer the control from one job to another.

- ▶ Problem
 - ▶ Limited memory
 - ▶ Speed mismatch
 - ▶ Different job requirement

JOB

Program
Input output data
Control instructions

Types Of Operating System

- ☐ Batch System
 - ☐ Parallel System
 - ☐ Interactive System
 - ☐ Distributed System
 - ☐ Time Sharing System
 - ☐ Clustered System
 - ☐ Real Time Sharing System
 - ☐ Handheld System
 - ☐ Network System
-

Batch Operating System

- The users of batch operating system do not interact with the computer directly.
 - Each user prepares his job on an off-line device like punch cards and submits it to the computer operator. To speed up processing, jobs with similar needs are batched together and run as a group.
 - Thus, the programmers left their programs with the operator. The operator then sorts programs into batches with similar requirements.
-

Batch Operating System

- The problems with Batch Systems are following.
 - Lack of interaction between the user and job.
 - CPU is often idle, because the speeds of the mechanical I/O devices is slower than CPU.
 - Difficult to provide the desired priority



courtesy Argonne National Laboratory

Interactive Operating System

- In an interactive operating system, the user interacts directly with the operating system to supply commands and data as the application program executes and
- the user receives the results of processing immediately. The user is in direct two way communication with the computer
- Most operating system is this type

Interactive Operating System

- There will be an user interface in place to allow this to happen. It could be CLI or GUI.

```
./development/web/thebuggenie $ ./dbg_cli remote:ls_issues thebuggenie -state=open -issuetype=bug
Querying http://thebuggenie for list of issues...

Filters:
State: open
Issuetype: bug
Assigned to: me

The following 12 issues were found:
Documentation request #294 - Missing documentation for delayed mailing
Enhancement #300 - Success taking alias (virtual directory)
Enhancement #300 - Add bug basic version to general tab of configuration page
Feature request #300 - List of open issues should somehow notify the user of any changes by others to it
Bug report #315 - Danish character "Å" "A.A.A." not working as file name on file upload
Enhancement #330 - Wiki searchfunction
Bug report #343 - Accept issue vs. Progress
Bug report #344 - Weekly completion
Enhancement #351 - General setting to remove govtar
Bug report #372 - email not sent
Feature request #376 - Configurable offline Page
Bug report #386 - Issue adding user/team to a project; user/team is always added with the settings from

If you are going to update or query any of these issues, use the
issue number shown in front of the issue (do not include the
issue type), ex:
./dbg_cli remote:update_issue projectname 300
./dbg_cli remote:show_issue projectname 300
./dbg_cli remote:ls_transitions projectname 300

or
./dbg_cli remote:update_issue projectname PREFIX-12
./dbg_cli remote:show_issue projectname PREFIX-12
./dbg_cli remote:ls_transitions projectname PREFIX-12
```

Time Sharing Operating System

- Time sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time.
- Time-sharing or multitasking is a logical extension of multiprogramming. Processor's time which is shared among multiple users simultaneously is termed as time-sharing.
- The main difference between Multiprogrammed Batch Systems and Time-Sharing Systems is that in case of Multiprogrammed batch systems, objective is to maximize processor use, whereas in Time-Sharing Systems objective is to minimize response time.



Time Sharing Operating System

- Multiple jobs are executed by the CPU by switching so frequently. Thus, the user can receive an immediate response.
 - Advantages
 - Provide advantage of quick response.
 - Avoids duplication of software.
 - Reduces CPU idle time.
 - Disadvantages
 - Problem of reliability.
 - Question of security and integrity of user programs and data.
 - Problem of data communication.
-



Real Time Operating System

- Provides environment for executing programs
 - Process abstraction for multitasking/concurrency
 - Scheduling
 - Hardware abstraction layer (device drivers)
 - File systems
 - Communication
-

Real Time Operating System

- Mass use in Electronic Devices.
- User Interface is very small

Such as X-ray, city –Scan,etc



Network Operating System

- Runs on a server and provides server the capability to manage data, users, groups, security, applications, and other networking functions.
- The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.
- Examples of network operating systems are Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD.



Network Operating System

- The advantages
 - Centralized servers are highly stable.
 - Security is server managed.
 - Upgrades to new technologies and hardware can be easily integrated into the system.
 - Remote access to servers is possible from different locations and types of systems.
 - The disadvantages
 - High cost of buying and running a server.
 - Dependency on a central location for most operations.
 - Regular maintenance and updates are required.
-



Parallel Operating System

- Parallel operating systems are used to interface multiple networked computers to complete tasks in parallel.
 - The architecture of the software is often a UNIX-based platform, which allows it to coordinate distributed loads between multiple computers in a network.
 - Parallel operating systems are able to use software to manage all of the different resources of the computers running in parallel, such as memory, caches, storage space, and processing power.
 - Parallel operating systems also allow a user to directly interface with all of the computers in the network
-

Parallel Operating System

- A parallel operating system works by dividing sets of calculations into smaller parts and distributing them between the machines on a network. To facilitate communication between the processor cores and memory arrays, routing software has to either share its memory by assigning the same address space to all of the networked computers, or distribute its memory by assigning a different address space to each processing core.
 - Scientists, researches, and industries often choose to use parallel operating systems because of its cost effectiveness as well. It costs far less money to assemble a parallel computer network than it costs to develop and build a super computer for research.
-

Distributed Operating System

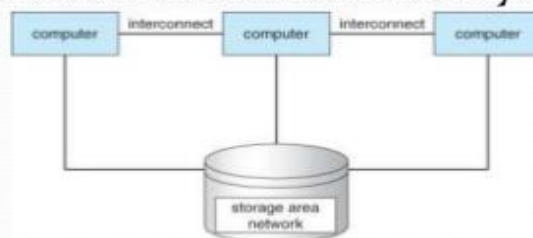
- Distributed systems use multiple central processors to serve multiple real time application and multiple users. Data processing jobs are distributed among the processors accordingly to which one can perform each job most efficiently.
 - The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines). These are referred as loosely coupled systems or distributed systems. Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, computers and so on.
-

Distributed Operating System

- The advantages
 - With resource sharing facility user at one site may be able to use the resources available at another.
 - Speedup the exchange of data with one another via electronic mail.
 - If one site fails in a distributed system, the remaining sites can potentially continue operating.
 - Better service to the customers.
 - Reduction of the load on the host computer.
 - Reduction of delays in data processing.
-

Clustered Operating System

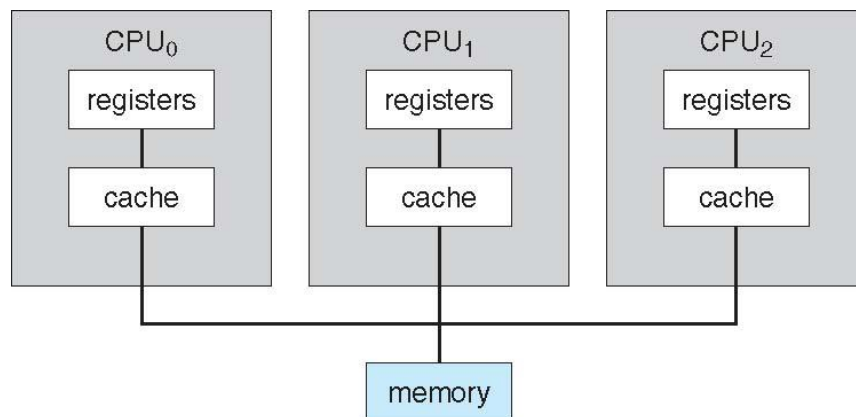
- Clustered computers share storage and are closely linked via **Local-area network (LAN)** or a faster interconnection.
Clustered System Combine the best feature of both distributed OS and Multi processor system
- A Group of connected computer working together called as one unit called clustered System



Handheld Operating System

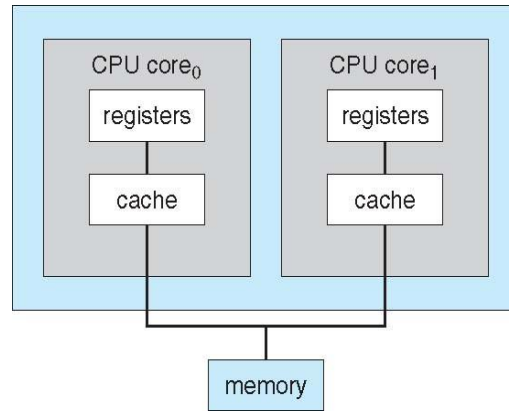
- Handheld OS known as Mobile Operating system such as Smartphone, tablet, PDA, or other mobile device.
- It's a combine features of a personal computer operating system with other features useful for mobile or handheld use; usually including, and most of the following considered essential in modern mobile systems; a touchscreen, cellular, Bluetooth, Wi-Fi, GPS mobile navigation, camera, video camera, speech recognition, voice recorder, music player, near field communication and infrared blaster.

Symmetric Multiprocessing Architecture



A Dual-Core Design

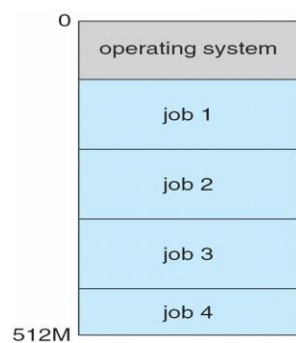
- Multi-chip and **multicore**
- Systems containing all chips
 - Chassis containing multiple separate systems



Operating System Structure

- **Multiprogramming (Batch system)** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program executing in memory ⇒ **process**
 - If several jobs ready to run at the same time ⇒ **CPU scheduling**
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory

Memory Layout for Multiprogrammed System



Operating-System Operations

- **Interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - ❖ Software error (e.g., division by zero)
 - ❖ Request for operating system service
 - ❖ Other process problems include infinite loop, processes modifying each other or the operating system

Operating-System Operations (cont.)

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - ▶ Provides ability to distinguish when system is running user code or kernel code
 - ▶ Some instructions designated as **privileged**, only executable in kernel mode
 - ▶ System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**

Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights

Chapter 2: Operating-System Structures

Objectives

- ✓ To describe the services an operating system provides to users, processes, and other systems
- ✓ To discuss the various ways of structuring an operating system
- ✓ To explain how operating systems are installed and customized and how they boot

Common Operating-System Components

- Process Management
- Main Memory Management
- File Management
- I/O System Management
- Secondary Management
- Protection System

Process Management

- A *process* is a program in execution. A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task
- The operating system is responsible for the following activities in connection with process management.
 - ✓ Process creation and deletion.
 - ✓ process suspension and resumption.
 - ✓ Provision of mechanisms for:
 - ❖ process synchronization
 - ❖ process communication

Main-Memory Management

- Memory is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device. It loses its contents in the case of system failure.
- The operating system is responsible for the following activities in connections with memory management:
 - Keep track of which parts of memory are currently being used and by whom.
 - Decide which processes to load when memory space becomes available.
 - Allocate and deallocate memory space as needed.

File Management

- A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data.
- The operating system is responsible for the following activities in connections with file management:
 - File creation and deletion.
 - Directory creation and deletion.
 - Support of primitives for manipulating files and directories.
 - Mapping files onto secondary storage.
 - File backup on stable (nonvolatile) storage media.

I/O System Management

- The I/O system consists of:
 - A buffer-caching system
 - A general device-driver interface
 - Drivers for specific hardware devices

Secondary-Storage Management

- Since main memory (*primary storage*) is volatile and too small to accommodate all data and programs permanently, the computer system must provide *secondary storage* to back up main memory.
- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.
- The operating system is responsible for the following activities in connection with disk management:
 - Free space management
 - Storage allocation
 - Disk scheduling

Protection System

- *Protection* refers to a mechanism for controlling access by programs, processes, or users to both system and user resources.
- The protection mechanism must:
 - distinguish between authorized and unauthorized usage.
 - specify the controls to be imposed.
 - provide a means of enforcement.

Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
 - ✓ **User interface** - Almost all operating systems have a user interface (UI).
 - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**.
 - ✓ **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - ✓ **I/O operations** - A running program may require I/O, which may involve a file or an I/O device

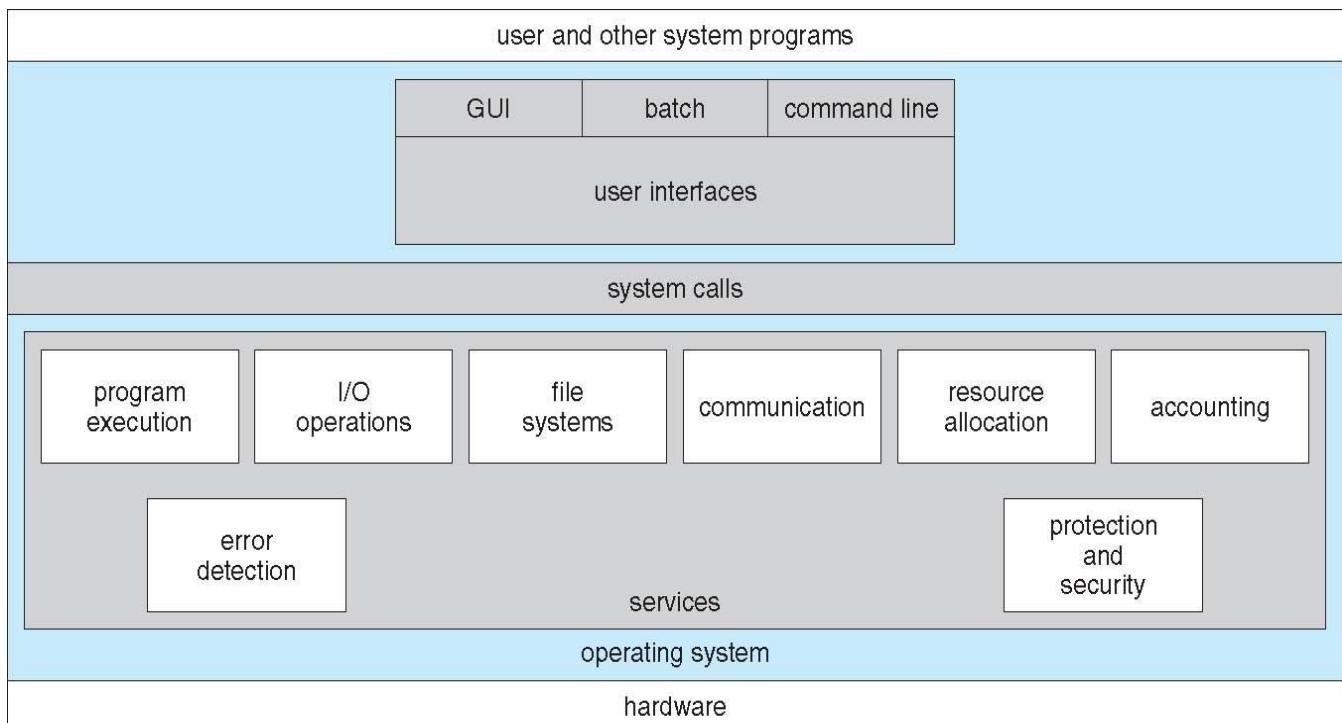
Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
 - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
 - **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - ▶ Communications may be via shared memory or through message passing (packets moved by the OS)
 - **Error detection** – OS needs to be constantly aware of possible errors
 - ▶ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ▶ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ▶ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - ▶ Many types of resources - CPU cycles, main memory, file storage, I/O devices.
 - **Accounting** - To keep track of which users use how much and what kinds of computer resources
 - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - ▶ **Protection** involves ensuring that all access to system resources is controlled
 - ▶ **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

A View of Operating System Services

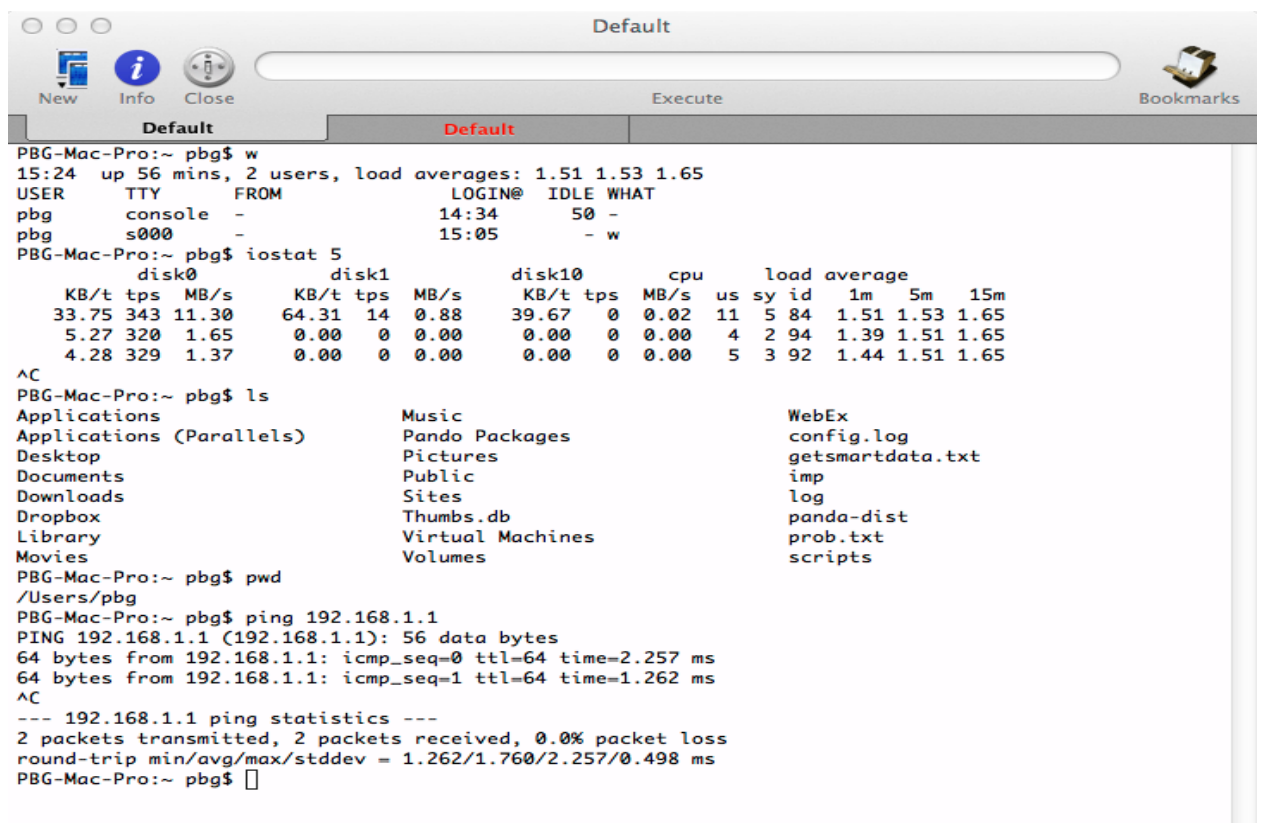


User Operating System Interface – CLI

CLI or **command interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
 - If the latter, adding new features doesn't require shell modification

Bourne Shell Command Interpreter



```
PBG-Mac-Pro:~ pbg$ w
15:24 up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER      TTY      FROM          LOGIN@      IDLE WHAT
pbg       console  -             14:34       50  -
pbg       s000    -             15:05       -  w
PBG-Mac-Pro:~ pbg$ iostat 5
          disk0      disk1      disk10      cpu      load average
KB/t tps MB/s    KB/t tps MB/s    KB/t tps MB/s  us sy id 1m 5m 15m
33.75 343 11.30    64.31 14  0.88    39.67 0  0.02  11  5 84 1.51 1.53 1.65
5.27 320  1.65     0.00 0  0.00     0.00 0  0.00   4  2 94 1.39 1.51 1.65
4.28 329  1.37     0.00 0  0.00     0.00 0  0.00   5  3 92 1.44 1.51 1.65
^C
PBG-Mac-Pro:~ pbg$ ls
Applications          Music                  WebEx
Applications (Parallels)  Pando Packages       config.log
Desktop               Pictures              getsmartdata.txt
Documents             Public                imp
Downloads             Sites                 log
Dropbox              Thumbs.db             panda-dist
Library              Virtual Machines     prob.txt
Movies               Volumes               scripts
PBG-Mac-Pro:~ pbg$ pwd
/Users/pbg
PBG-Mac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBG-Mac-Pro:~ pbg$
```

User Operating System Interface – GUI

- ✓ User-friendly **desktop** metaphor interface
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))

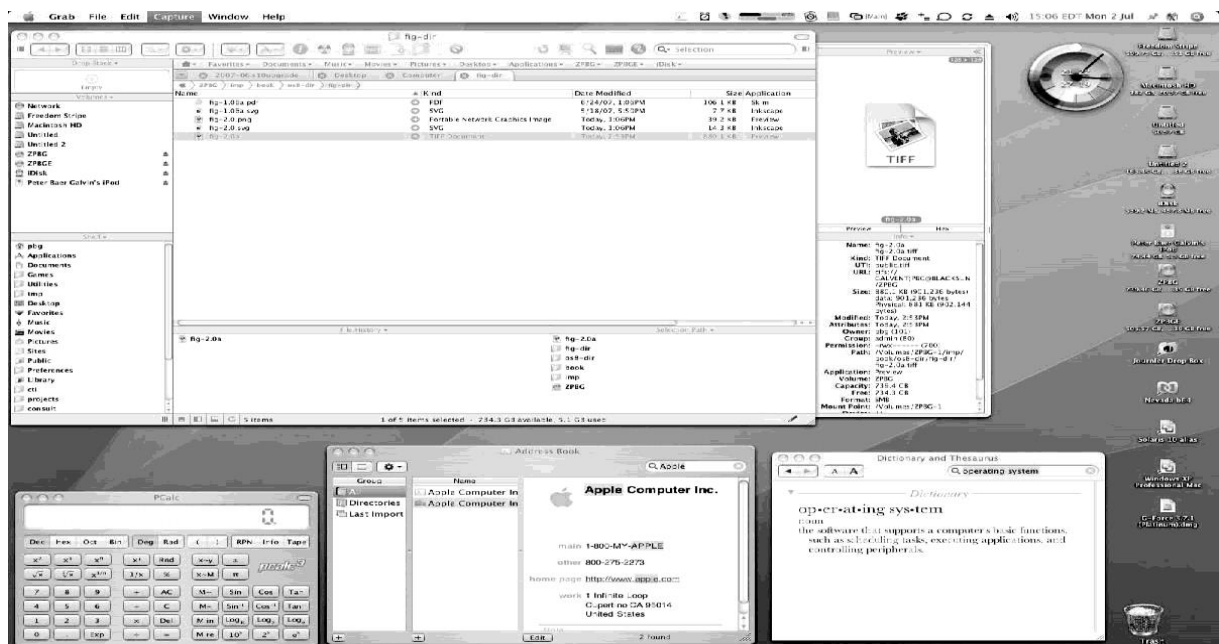
- Invented at Xerox PARC
- ✓ Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

Touchscreen Interfaces

- Touchscreen devices require new interfaces
 - Mouse not possible or not desired
 - Actions and selection based on gestures
 - Virtual keyboard for text entry
- Voice commands.



The Mac OS X GUI



System Calls

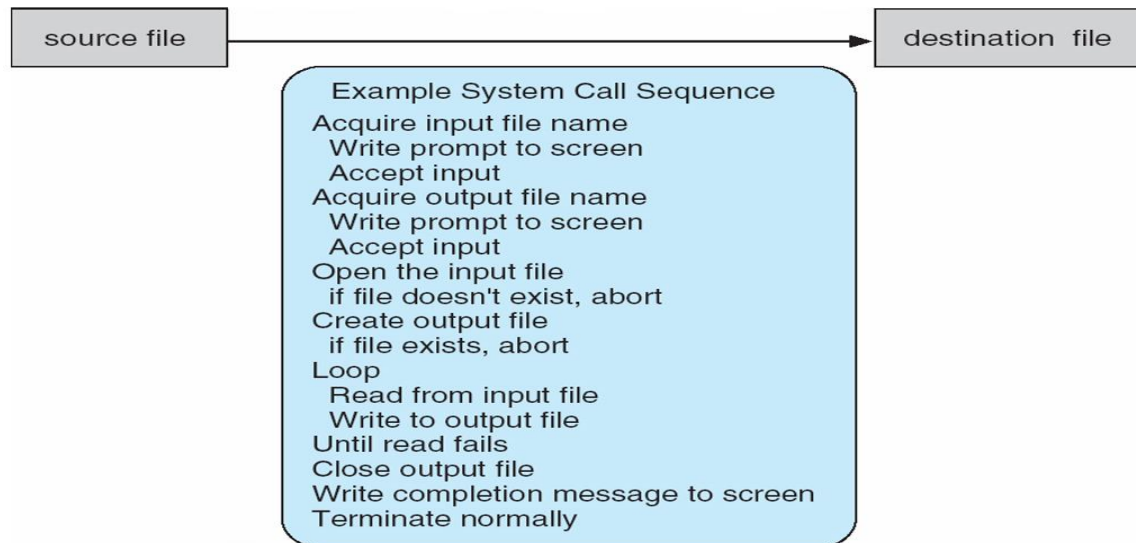
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use

- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic

Example of System Calls

- System call sequence to copy the contents of one file to another file



Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t      read(int fd, void *buf, size_t count)
```

return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

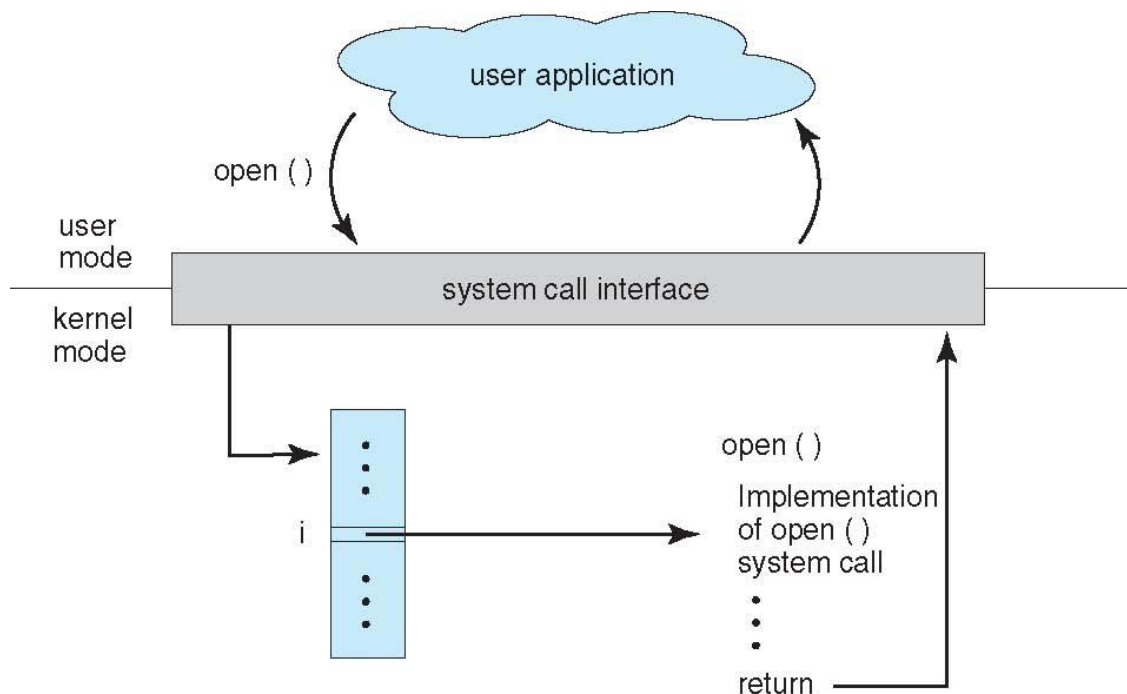
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

System Call Implementation

- Typically, a number associated with each system call
 - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)

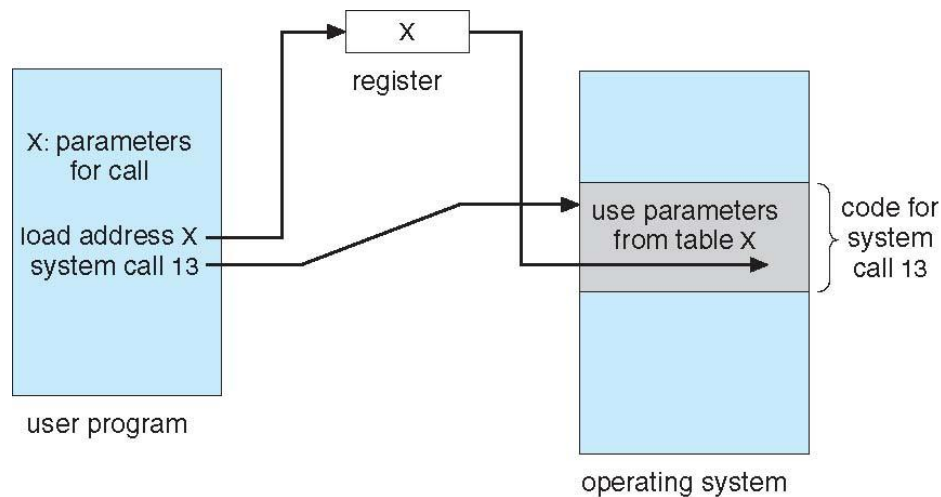
API – System Call – OS Relationship



System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in registers
 - In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - This approach taken by Linux and Solaris
 - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls

- Process control
 - create process, terminate process
 - end, abort
 - load, execute
 - get process attributes, set process attributes
 - wait for time
 - wait event, signal event
 - allocate and free memory
 - Dump memory if error
 - **Debugger** for determining **bugs**, **single step** execution
 - **Locks** for managing access to shared data between processes

Types of System Calls

- File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes
 - logically attach or detach devices

Types of System Calls (Cont.)

- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages if **message passing model** to **host name** or **process name**
 - From **client** to **server**

- **Shared-memory model** create and gain access to memory regions
- transfer status information
- attach and detach remote devices

Types of System Calls (Cont.)

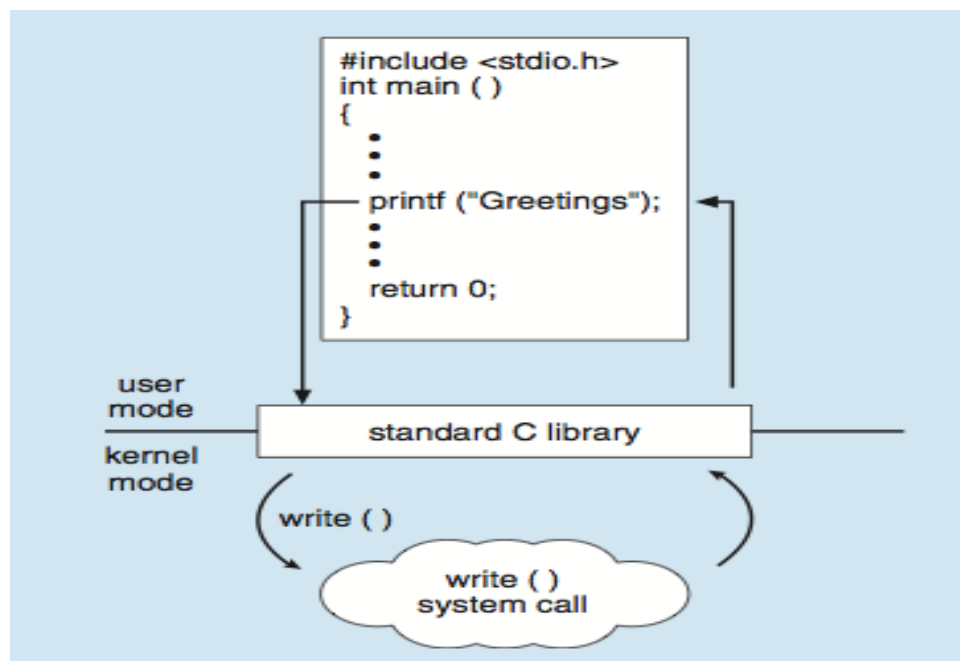
- Protection
 - Control access to resources
 - Get and set permissions
 - Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

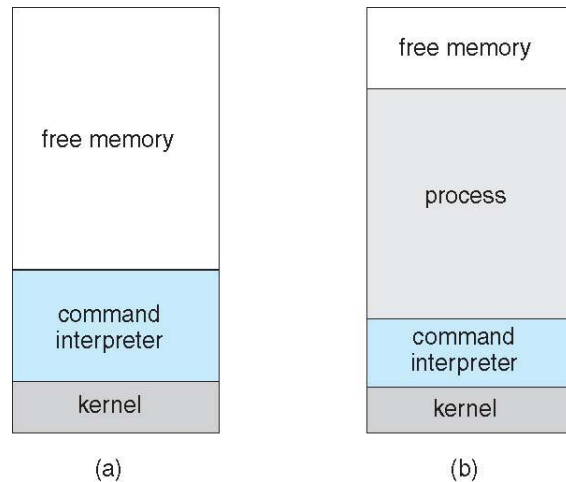
Standard C Library Example

- C program invoking printf() library call, which calls write() system call



Example: MS-DOS

- Single-tasking
- Shell invoked when system booted
- Simple method to run program
 - No process created
- Single memory space
- Loads program into memory, overwriting all but the kernel
- Program exit -> shell reloaded

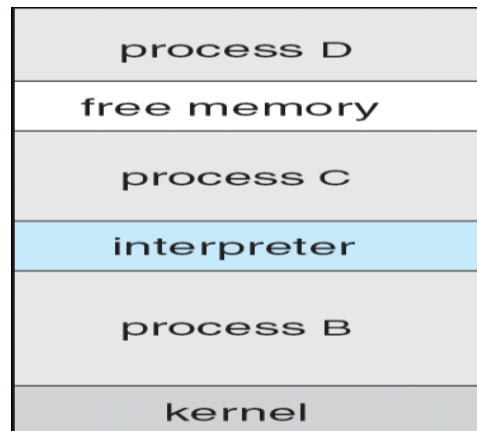


At system startup

running a program

Example: FreeBSD

- ✓ Unix variant
- ✓ Multitasking
- ✓ User login -> invoke user's choice of shell
- ✓ Shell executes fork() system call to create process
 - Executes exec() to load program into process
 - Shell waits for process to terminate or continues with user commands
- ✓ Process exits with:
 - code = 0 – no error
 - code > 0 – error code



System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
 - File manipulation
 - Status information sometimes stored in a File modification
 - Programming language support
 - Program loading and execution
 - Communications
 - Background services
 - Application programs
- Most users' view of the operating system is defined by system programs, not the actual system calls

System Programs

- Provide a convenient environment for program development and execution
 - Some of them are simply user interfaces to system calls; others are considerably more complex
 - **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information
 - Typically, these programs format and print the output to the terminal or other output devices
 - Some systems implement a **registry** - used to store and retrieve configuration information

System Programs (Cont.)

- **File modification**
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another.

System Programs (Cont.)

- **Background Services**
 - Launch at boot time
 - Some for system startup, then terminate
 - Some from system boot to shutdown
 - Provide facilities like disk checking, process scheduling, error logging, printing
 - Run in user context not kernel context
 - Known as **services, subsystems, daemons**
- **Application programs**
 - Don't pertain to system
 - Run by users
 - Not typically considered part of OS
 - Launched by command line, mouse click, finger poke

Operating System Design and Implementation

- ✓ Design and Implementation of OS not “solvable”, but some approaches have proven successful
- ✓ Internal structure of different Operating Systems can vary widely
- ✓ Start the design by defining goals and specifications
- ✓ Affected by choice of hardware, type of system
- ✓ **User** goals and **System** goals

- User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
- System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

Operating System Design and Implementation (Cont.)

- ✓ Important principle to separate
 - **Policy:** *What* will be done?
 - **Mechanism:** *How* to do it?
- ✓ Mechanisms determine how to do something, policies decide what will be done
- ✓ The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer)
- ✓ Specifying and designing an OS is highly creative task of **software engineering**

Implementation

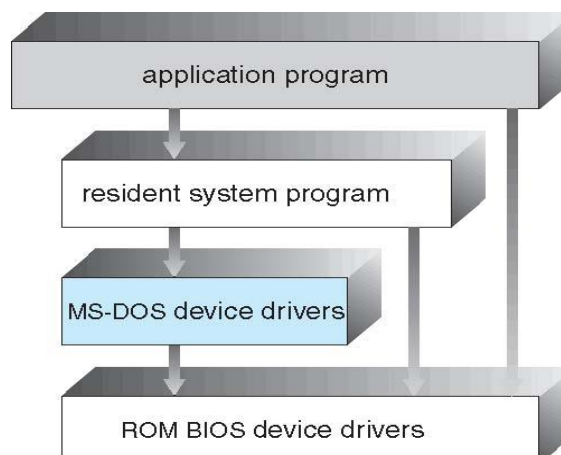
- ✓ Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- ✓ Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- ✓ More high-level language easier to **port** to other hardware
 - But slower
- ✓ **Emulation** can allow an OS to run on non-native hardware

Operating System Structure

- ✓ General-purpose OS is very large program
- ✓ Various ways to structure ones
 - Simple structure – MS-DOS
 - More complex -- UNIX
 - Layered – an abstraction
 - Microkernel –Mach

Simple Structure -- MS-DOS

- ✓ MS-DOS – written to provide the most functionality in the least space
 - Not divided into modules
 - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



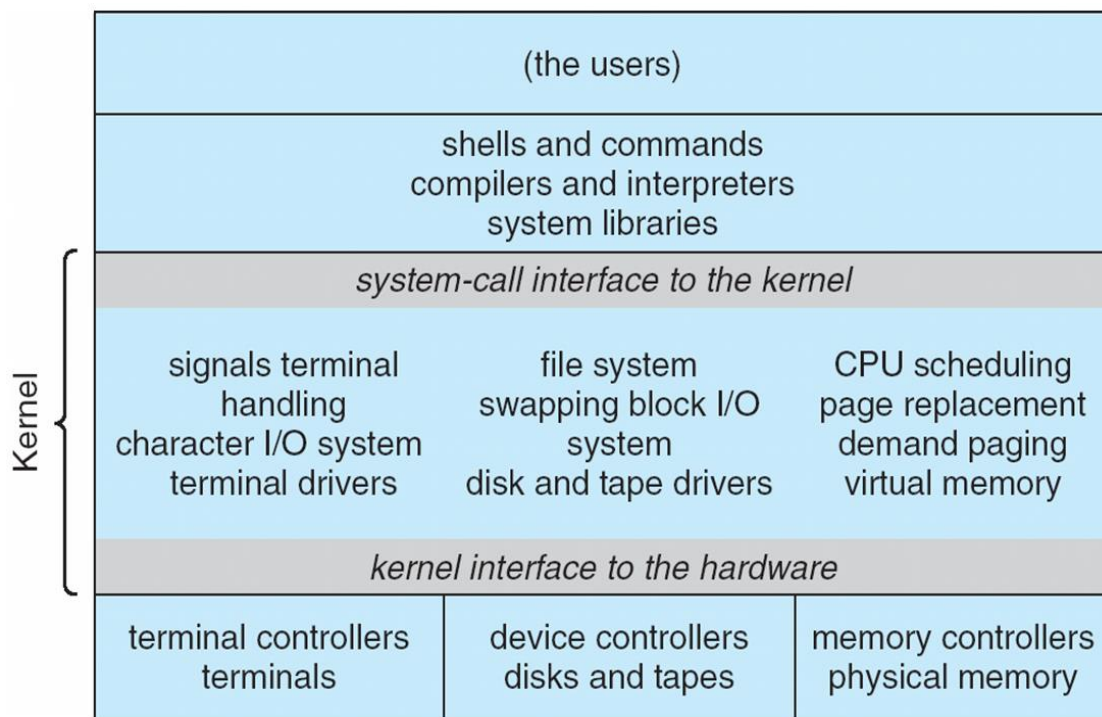
Non Simple Structure -- UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs
- The kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

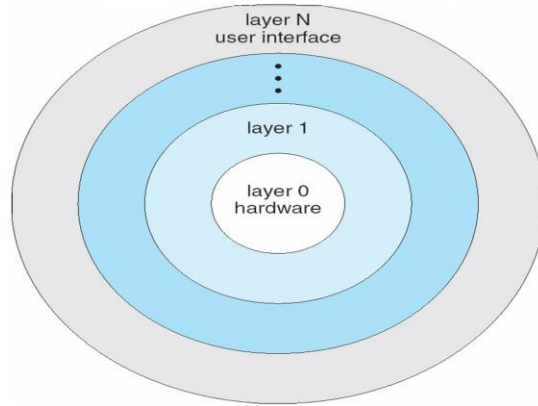
Traditional UNIX System Structure

Beyond simple but not fully layered



Layered Approach

- ✓ The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- ✓ With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Microkernel System Structure

- ✓ Moves as much from the kernel into user space
- ✓ **Mach** example of **microkernel**
 - Mac OS X kernel (**Darwin**) partly based on Mach
- ✓ Communication takes place between user modules using **message passing**
- ✓ Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- ✓ Detriments:
 - Performance overhead of user space to kernel space communication

Microkernel System Structure

